Computers can only store 0s and 1s. So far, we've only stored positive binary numbers (unsigned). How do we store negative values (signed)

unsigned binary number: the leftmost bit is the most significant bit.

↑ carries the most mathematical weight.

Signed binary number: the leftmost bit is the sign bit

- 0 = positive
- 1 = negative

unsigned (11001) = $\underset{16}{1}\ \underset{8}{1}\ \underset{4}{0}\ \underset{2}{0}\ \underset{1}{1}$ = 25

signed (11001) = $\underset{(-)}{1}\ \underset{8}{1}\ \underset{4}{0}\ \underset{2}{0}\ \underset{1}{1}$ = -9

} same bits, different readings

For signed binary, we use 8 bits to represent numbers

+9 can only be represented as 00001001

-9 can be represented in 3 ways

1) Signed Magnitude: 00001001 (+9) -> 10001001 (-9)
   - Change the sign bit from positive (0) to negative (1)

2) Signed 1's comp: 00001001 (+9) -> 11110110 (-9)
   - Take the 1's complement of 9 in binary

3) Signed 2's comp: 00001001 (+9) -> 11110111 (-9)
   - Take the 2's complement of 9 in binary

All these methods result in binary numbers with a signed bit of 1. So, these 3 are only ever used to make negatives.

Ex: Find the 2's complement of 4

$4 = (00000100)_2 = 11111011 + 1 = 11111100$

Represent 4 as a signed 2's complement

Because signed 2's complement only represents a negative number, positive numbers are represented normally.

$4 = (00000100)_2 = 00000100$

Signed Addition (2's complement)

1) Represent each number in their signed 2's complement (8-bits)
2) Add the two numbers (including their signed bits).
3) Throw away any carry over. It doesnt tell us anything

Ex: 6 = 00000110 ——————> + 00000110

\+ -13 00001101 -> 11110010 + 1 -> 11110011

-7 11111001

To confirm, negative answers. take the 2's comp of the sum. You should get the same integer value. just in positive form.

11111001 -> 00000110 + 1 = 00000111 = 7

Ex: -6 = 00000110 -> 11111001 + 1 -> 11111010

+13 00001101 ——————> + 00001101

7 00000111

we have a carry over of 1, so we discard it. The 0 means the answer is positive

Signed Subtraction (2's complement)

1) Represent each number in their signed 2's complement (8-bits)
2) Take the 2's complement of the number being subtracted
3) Add it back instead.
4) Throw away any carry over

Ex: -6 - (-13)

1) -6 in signed 2's comp representation = 00000110 -> 11111001 + 1 = 11111010

-13 in signed 2's comp representation = 00001101 -> 11110010 + 1 = 11110011

2) 2's comp of 11110011 = 00001100 + 1 = 00001101

3)
```
  1 1 1 1 1
  11111010
+ 00001101
----------
  00000111
```
Discard the carry

↑ The subtraction equals positive 7.

Computers store everything as 1s or 0s, but those 0s and 1s dont always represent a binary number. Sometimes they are codes for decimal digits, letters, or symbols.

Binary code: a pattern of 0s and 1s that mean something

Key Rule: n bits can create 2^n different bit patterns.

↳ Lets say we want to label "+", "−", "x", "÷" using 0s and 1s.

We want to create four unique patterns using as little bits as possible.

+ = 0
x = 1
÷ = ?
− = ?

} use all possible options / still not assigned

+ = 0 0
x = 0 1
÷ = 1 0
− = 1 1

2 bits can create $(2^2 = 4)$ unique combinations of 1s and 0s

3 bits can create $(2^3 = 8)$ unique combinations of 1s and 0s

Binary Coded Decimal (BCD): humans think in decimals, so the computer must account for that each decimal digit gets its own 4-bit code.

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

$(396)_{10}$ in BCD = 3 9 6

$(0011\ 1001\ 0110)_{BCD}$

$(185)_{10}$ in BCD = 1 8 5

$(0001\ 1000\ 0101)_{BCD}$

For BCD, any number greater than 9 is invalid.

BCD is not the same as binary.

$(185)_{10}$ -> (0001 1000 0101)$_{BCD}$ } BCD uses more bits

$(185)_{10}$ -> $(10111001)_2$

Binary Addition: 1) Add the BCD digits as if they were binary

2) If the sum is less than or equal to 9, leave it

3) if the sum is greater than 9, or produces a carry over, add 0110 (6) to fix

Ex: 4 = 0100 Ex: 4 = 0100 1100

\+ 5 = 1010 + 8 = 1000 -> 0110

9 1001 12 1100 ...10010

↑ The +6 intentionally creates another BCD term (0001)

What if we have multidigit addition?

Ex: 184 = 0001 1000 0100

\+ 576 = 0101 0111 0110

760 0110 10000 1010 ← greater than 9, add 6

\+ 0110 + 0110

0110 0000

7 6 0

Decimal Digit	BCD 8421	2421	Excess-3	8, 4, −2, −1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
Unused bit combi-nations	1010	0101	0000	0001
	1011	0110	0001	0010
	1100	0111	0010	0011
	1101	1000	1101	1100
	1110	1001	1110	1101
	1111	1010	1111	1110

These are other examples of binary codes

- 2421 = weighted code with different weight positions
- Excess-3 = each decimal digit + 3
- 84-2-1 = weighted code with different weight positio

Binary Logic: binary variables and logic operators

variables can either = 0 (F\off) or 1 (T\on)

Three logic operators:

1) **AND**: output (z) is 1 when both x and y = 1

$x \cdot y$, $x(y)$, xy

2) **OR**: output (z) is 1 when either x or y = 1.

$x + y$

3) **NOT**: output (z) is the negation \ complement of binary variable

$\bar{x}$, x'

Truth Tables and Logic Gates:

AND

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	z
0	0	0
0	1	1
1	0	1
1	1	1

NOT

x	z
0	1
1	0